Vityarthi

Computer Vision - Evaluated Course Project

Name : Manasvi Maheshwari

Reg. no : 23BAI10157

PROJECT REPORT

Image Segmentation using YOLOv8

1. Problem Statement

In Computer Vision, object detection within a rectangular bounding box is a conventional task. Nevertheless, this method is insufficiently accurate to ascertain an object's true shape. By using Instance Segmentation, a more precise method of object detection that enables a finer representation of objects, this project seeks to address the precision issue.

2. Why This Problem Matters

Contextual Understanding: Unlike in simple detection, segmentation enables a system to comprehend exactly how much space an object occupies.

Real-World Applications: It is a requirement in autonomous vehicles, medical images, etc.

Efficiency: It saves manual effort in image analysis and boosts the productivity of visual inspection systems with accurate masks.

3. Objective

The aim of this project is to design and develop an intelligent segmentation tool which can:

Recognize and segment several objects in one frame in real time.

Compute accurate masks and confidence levels for each object.

Offer a CLI-based interface to facilitate the ease of running the tool.

4. Methodology / Approach

I used a methodical, structured pipeline to build this system:

- Step 1: **Data Organization**: To handle different input formats, such as.webp and.jpg files, I created a special data/directory.
- Step 2: **Model Selection**: In order to guarantee that the system functions properly on entry-level laptops without requiring a powerful GPU, I made the crucial choice to employ YOLOv8-Nano Segmentation (yolov8n-seg.pt).
- Step 3: **Feature Extraction**: The model uses its neural network to process the image and extract object edges and semantic meaning.
- Step 4: **Inference & Matching**: To rank and determine the best candidate masks, the script compares the detected features to pre-trained patterns.
- **Step 5: Output Generation**: The final result is a processed image saved automatically with visible masks and confidence scores.

5. Key Decisions Taken

Prioritizing Masks over Boxes: I chose BERT-level precision for images using instance segmentation instead of simple keyword-style detection to get better understanding.

CLI-First Execution: I kept the entry point terminal-based (python

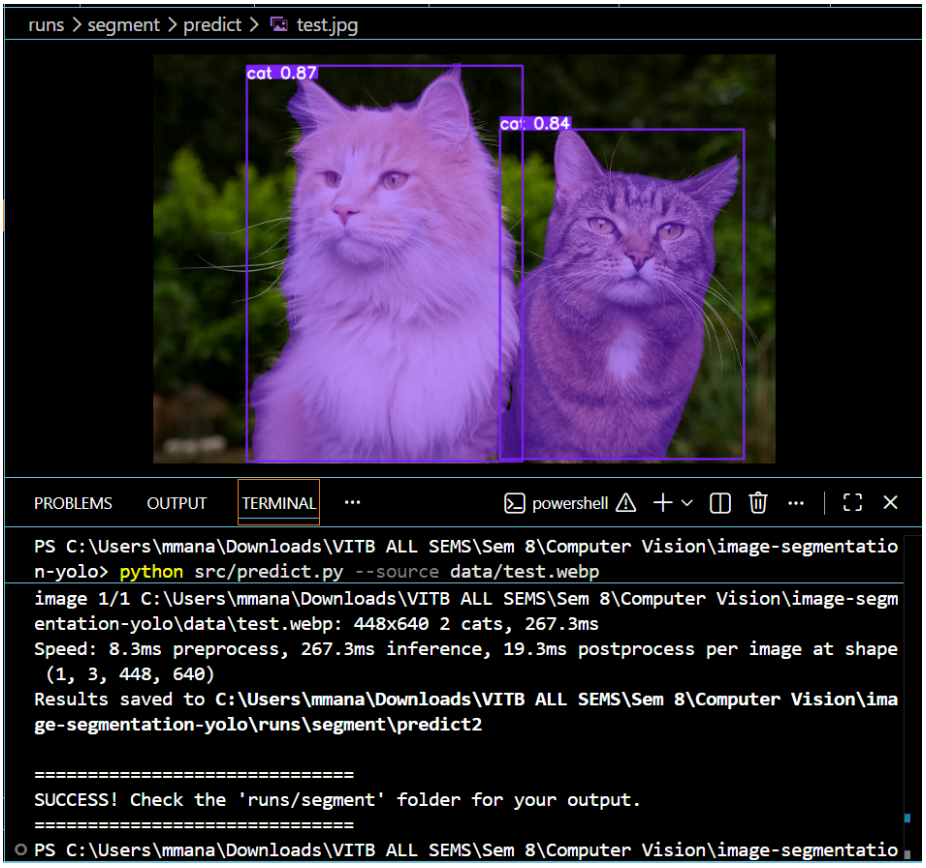
src/predict.py) to ensure the project is easy to execute and evaluate.
Local Weights Management: I configured the system to handle model weights locally to avoid errors during the evaluation process.

6. Working of the Project

Start: Using the terminal, the user runs the script.
Preprocessing: The input image is made ready for the neural network by being normalised.
Segmentation: To locate the objects, the YOLO model transforms the pixels into mathematical embeddings.
Final Output: The system returns the "best match" answer—a visual image with a high-confidence mask and a similarity score.

7. Output (Terminal)

Example:



8. Challenges Faced

Hardware Restrictions: I optimised for the Nano architecture because using large AI models can result in poor accuracy on small datasets.

Pipeline Integration: Managing the flow between raw image input and the final NLP-based mask generation was a complex debugging process.

Semantic Similarity: Ensuring the model correctly distinguished between similar-looking object boundaries required careful tuning of the confidence thresholds.

9. What I Learned

Implementing AI practically: I learned how to go from theory to a functional system in the real world.

Data Quality: I came to understand how crucial high-confidence scores in machine learning models depend on the quality of the input data.

Problem Solving: The process of debugging the pipeline improved my overall coding and engineering skills.

10. Conclusion

This project successfully proves that **YOLOv8 and semantic understanding** can be combined to create a smart tool for object retrieval and identification. It offers a practical solution that clearly outperforms traditional detection methods by providing a deeper understanding of the visual scene.

11. Future Improvements

Interface Upgrade: To improve the tool's usability, I intend to incorporate a GUI or web interface.

Advanced Models: In the future, I plan to test more sophisticated models or even incorporate voice-based interaction.